

Tutorial 8 — Exercise 2: The F -Test in R

Introductory Econometrics — TA Session

Duration: 50 minutes | Based on Exercise 1

In Exercise 1 you derived the F -statistic on paper. Now you will compute it from scratch in R, verify it against built-in functions, and see the test make the right decision.

We simulate data from the same wage equation as Lecture 14. The key advantage of simulation: because we choose the true parameters, we *know* whether H_0 is true or false — something we never know with real data. This lets us check whether the test behaves as the theory predicts.

Step 1. Generate the data.

We create a dataset of $n = 500$ workers with the same variables as the Lecture 14 wage equation.

```
rm(list = ls())      # start with a clean environment
set.seed(326)        # fix the random seed so results are reproducible
n <- 500             # sample size: 500 workers
```

First, we generate the regressors. Each line creates one variable for all n individuals at once:

```
# runif(n, a, b) draws n values uniformly between a and b
Exp      <- runif(n, 1, 30)      # current job experience: between 1 and 30 years
Exp2     <- Exp^2                # experience squared (for diminishing returns)

PrevExp   <- runif(n, 0, 20)     # previous experience: between 0 and 20 years
PrevExp2  <- PrevExp^2           # previous experience squared

# sample(x, n, replace = TRUE) draws n integers from the set x with replacement
Educ      <- sample(8:20, n, replace = TRUE) # education: 8 to 20 years
```

Next, we set the **true population parameters**. This is the “God’s-eye view” — we decide exactly how wages are determined. In real research you never know these; here we choose them so we can verify the test.

```
b0 <- 0.5          # intercept
b1 <- 0.04         # return to current experience (positive)
b2 <- -0.0008      # diminishing returns to experience (negative quadratic)
b3 <- 0            # <-- previous experience linear effect: ZERO
b4 <- 0            # <-- previous experience quadratic effect: ZERO
b5 <- 0.09         # return to education
sigma <- 0.4       # standard deviation of the error term
```

Design choice: H_0 is TRUE by construction

We set $\beta_3 = 0$ and $\beta_4 = 0$. This means previous experience has **no effect whatsoever** on wages in the population. The null hypothesis $H_0: \beta_3 = 0, \beta_4 = 0$ is true.

What should we expect? If the F -test works correctly, it should *fail to reject* H_0 about 95% of the time (at $\alpha = 0.05$). The other 5% of the time, it will reject even though H_0 is true — that is a Type I error, and it happens by design. The whole point of setting $\alpha = 0.05$ is accepting a 5% false rejection

```
rate.
```

Finally, generate the outcome variable:

```
# rnorm(n, mean, sd) draws n values from a normal distribution N(mean, sd^2)
# Each draw represents the unobserved factors affecting worker i's wage
U <- rnorm(n, mean = 0, sd = sigma)

# The data-generating process: this is the TRUE model
# Since b3 = b4 = 0, the PrevExp terms contribute nothing
lnWage <- b0 + b1*Exp + b2*Exp2 + b3*PrevExp + b4*PrevExp2 + b5*Educ + U

# Combine everything into a data frame for use with lm()
wages <- data.frame(lnWage, Exp, Exp2, PrevExp, PrevExp2, Educ)
```

Step 2. Estimate the unrestricted model and extract SSR_{ur} .

The unrestricted model includes all five regressors — it does not impose H_0 . This is the full model from Exercise 1, Step 2.

```
unrestricted <- lm(lnWage ~ Exp + Exp2 + PrevExp + PrevExp2 + Educ,
                  data = wages)
summary(unrestricted)
```

```
##
## Call:
## lm(formula = lnWage ~ Exp + Exp2 + PrevExp + PrevExp2 + Educ,
##     data = wages)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.45814 -0.26095 -0.00492  0.26473  1.17865
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.6004039  0.1048667   5.725 1.79e-08 ***
## Exp          0.0275052  0.0089785   3.063 0.00231 **
## Exp2        -0.0005383  0.0002824  -1.906 0.05724 .
## PrevExp     -0.0174051  0.0127518  -1.365 0.17290
## PrevExp2     0.0006716  0.0006089   1.103 0.27057
## Educ         0.0969909  0.0048850  19.855 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3853 on 494 degrees of freedom
## Multiple R-squared:  0.4757, Adjusted R-squared:  0.4704
## F-statistic: 89.66 on 5 and 494 DF,  p-value: < 2.2e-16
```

Reading the output

Look at the coefficients on `PrevExp` and `PrevExp2`. Since $\beta_3 = \beta_4 = 0$ in the DGP, their estimated coefficients should be close to zero (but not exactly zero — there is always sampling noise). Their individual p -values may or may not be below 0.05. Remember from Exercise 1, part (a): individual t -tests are *not* the right way to test a joint hypothesis.

Now extract SSR_{ur} . The function `resid()` returns the vector of OLS residuals $\hat{U}_i = Y_i - \hat{Y}_i$ for each observation. Squaring each one and summing gives $SSR_{ur} = \sum_{i=1}^n \hat{U}_i^2$:

```
SSR_ur <- sum(resid(unrestricted)^2)      # sum of squared residuals
df_ur  <- unrestricted$df.residual      # residual df = n - k - 1 = 500 - 5 - 1 = 494

cat("SSR_ur =", round(SSR_ur, 4), "\n")

## SSR_ur = 73.3532

cat("Residual df (n - k - 1) =", df_ur, "\n")

## Residual df (n - k - 1) = 494
```

Step 3. Estimate the restricted model and extract SSR_r .

Under $H_0: \beta_3 = 0, \beta_4 = 0$, we drop `PrevExp` and `PrevExp2` from the model. This is the restricted model from Exercise 1, Step 2.

```
restricted <- lm(lnWage ~ Exp + Exp2 + Educ, data = wages)

SSR_r <- sum(resid(restricted)^2)
cat("SSR_r =", round(SSR_r, 4), "\n")

## SSR_r = 73.7413

Now verify the key property from Exercise 1, part (b): imposing restrictions can never improve the fit.

cat("SSR_r >= SSR_ur?", SSR_r >= SSR_ur, "\n")

## SSR_r >= SSR_ur? TRUE

cat("Fit cost (SSR_r - SSR_ur) =", round(SSR_r - SSR_ur, 4), "\n")

## Fit cost (SSR_r - SSR_ur) = 0.3881
```

Interpreting the fit cost

The number $SSR_r - SSR_{ur}$ is the **total increase in the sum of squared residuals** when we drop `PrevExp` and `PrevExp2`. Since H_0 is true in our DGP (these variables genuinely have zero coefficients), the fit cost should be **small** — just sampling noise. If this number were large, it would suggest the excluded variables were genuinely important.

Step 4. Compute the F -statistic by hand.

Now we apply the formula from Exercise 1, Step 3:

$$F = \frac{(SSR_r - SSR_{ur}) / q}{SSR_{ur} / (n - k - 1)}$$

```
q <- 2      # number of restrictions (beta_3 = 0 AND beta_4 = 0)

# Numerator: average fit cost per restriction
numerator <- (SSR_r - SSR_ur) / q

# Denominator: mean squared error from unrestricted model (estimate of sigma^2)
denominator <- SSR_ur / df_ur

# F-statistic: signal-to-noise ratio
```

```
F_manual <- numerator / denominator

cat("Numerator (fit cost per restriction):", round(numerator, 4), "\n")

## Numerator (fit cost per restriction): 0.1941

cat("Denominator (MSE, estimate of sigma^2):", round(denominator, 4), "\n")

## Denominator (MSE, estimate of sigma^2): 0.1485

cat("F-statistic:", round(F_manual, 4), "\n")

## F-statistic: 1.3069
```

What does this number mean?

The F -statistic tells you how many times larger the fit loss per restriction is compared to the background noise. If $F \approx 1$, the fit loss is about what you would expect from random variation alone — no evidence against H_0 . If F is much larger than 1, the fit loss is too big to be explained by chance. In our DGP where H_0 is true, F should be modest (close to 1 on average).

Step 5. Compute the p -value and make a decision.

We need two R functions for the F -distribution. They work exactly like `pnorm/qnorm` from Tutorial 7, but for the F -distribution instead of the normal:

Two essential R functions

`qf(p, df1, df2)` answers: “what F -value has fraction p of the $F_{df1, df2}$ distribution below it?” This gives the **critical value**.

Example: `qf(0.95, df1 = 2, df2 = 494)` returns the value such that 95% of the $F_{2, 494}$ distribution lies below it. If your F -statistic exceeds this value, you reject H_0 at $\alpha = 0.05$.

`pf(x, df1, df2, lower.tail = FALSE)` answers: “what fraction of the $F_{df1, df2}$ distribution lies *above* x ?” This gives the **p -value**.

Always use `lower.tail = FALSE`. Without it, `pf()` returns the area *below* x , which is the wrong tail. A large F would give a p -value near 1 instead of near 0.

```
alpha <- 0.05

# Critical value: the 95th percentile of the F(2, 494) distribution
F_crit <- qf(1 - alpha, df1 = q, df2 = df_ur)

# p-value: probability of observing F >= F_manual under H0
# lower.tail = FALSE gives the UPPER tail probability
p_val <- pf(F_manual, df1 = q, df2 = df_ur, lower.tail = FALSE)

cat("Critical value (5%):", round(F_crit, 4), "\n")

## Critical value (5%): 3.014

cat("p-value:", round(p_val, 4), "\n")

## p-value: 0.2716
```

Now make the decision — exactly as described in Exercise 1, Step 5:

```
cat("F-statistic:", round(F_manual, 4), "\n")

## F-statistic: 1.3069

cat("Critical value:", round(F_crit, 4), "\n\n")

## Critical value: 3.014

if (F_manual > F_crit) {
  cat("DECISION: Reject H0 (F =", round(F_manual, 3),
      "> critical value =", round(F_crit, 3), ")\n")
  cat("Since H0 is TRUE in our DGP, this is a Type I error.\n")
  cat("This happens about 5% of the time --- exactly what alpha = 0.05 means.\n")
} else {
  cat("DECISION: Fail to reject H0 (F =", round(F_manual, 3),
      "<= critical value =", round(F_crit, 3), ")\n")
  cat("Correct decision --- H0 is indeed true in our DGP.\n")
  cat("The test correctly concluded that PrevExp does not affect wages.\n")
}

## DECISION: Fail to reject H0 (F = 1.307 <= critical value = 3.014 )
## Correct decision --- H0 is indeed true in our DGP.
## The test correctly concluded that PrevExp does not affect wages.
```

Step 6. Verify against linearHypothesis().

The `car` package provides `linearHypothesis()`, which computes the F -statistic automatically from the unrestricted model. Your manual calculation from Steps 4–5 should match exactly.

```
library(car)

# IMPORTANT: check what names R assigned to the coefficients
# The names you pass to linearHypothesis() must match these EXACTLY
names(coef(unrestricted))

## [1] "(Intercept)" "Exp"          "Exp2"          "PrevExp"       "PrevExp2"
## [6] "Educ"

# Test H0: beta_3 = 0 AND beta_4 = 0
linearHypothesis(unrestricted, c("PrevExp = 0", "PrevExp2 = 0"))

##
## Linear hypothesis test:
## PrevExp = 0
## PrevExp2 = 0
##
## Model 1: restricted model
## Model 2: lnWage ~ Exp + Exp2 + PrevExp + PrevExp2 + Educ
##
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1      496 73.741
## 2      494 73.353   2   0.38811 1.3069 0.2716
```

Reading the linearHypothesis() output

The output shows a table with two rows:

- **Row 1** (Model 1): the *restricted* model. `Res.Df = 496` is the residual df ($n - 3 - 1$). $RSS = SSR_r$.
- **Row 2** (Model 2): the *unrestricted* model. `Res.Df = 494` is the residual df ($n - 5 - 1$). $RSS = SSR_{ur}$.

Check three things:

1. The `F` value matches your `F_manual` from Step 4.
2. The `Pr(>F)` value matches your `p_val` from Step 5.
3. The difference in `Res.Df` is $496 - 494 = 2 = q$, the number of restrictions.

Common mistake: the variable names inside `c("PrevExp = 0", ...)` must *exactly* match the names from `names(coef(unrestricted))`. If your data frame used different column names (e.g., `prev_exp` instead of `PrevExp`), R would use those names instead, and you would need to match them. A typo produces an error, not a wrong answer — so read the error message carefully.

Step 7 (take-home). What happens when H_0 is false?

Go back to Step 1 and change the true parameters so that previous experience matters:

```
b3 <- 0.015      # positive linear effect (was 0)
b4 <- -0.0003    # diminishing returns (was 0)
```

Then re-run Steps 2–6. You should observe:

- $SSR_r - SSR_{ur}$ is now **much larger** (dropping relevant variables hurts the fit substantially).
- The F -statistic is **large** (well above the critical value).
- The p -value is **tiny** ($p \ll 0.05$).
- The test **correctly rejects** H_0 .

This is the **power** of the test: its ability to detect a false null. The larger the true effect (β_3, β_4), the larger F becomes and the more likely we are to reject. Try different values of `b3` and `b4` to see how the F -statistic responds.